Лекция 4: Принципы SOLID (Часть 1). Single Responsibility (SRP) и Open/Closed (OCP). Cohesion и Coupling

Курс: Объектно-ориентированное программирование 1 (CS2203)
Университет: КазНУ им. аль-Фараби
Преподаватель: Санжар Аубакиров, PhD
Аудитория: Бакалавриат, 2-й курс, специальность «6B06110 Программная инженерия»
Формат: Лекция, 2 академических часа (90 минут)
Методологический нарратив: Structure → Order (От структурных связей к инженерной дисциплине SOLID)

Введение и инженерный контекст

На первых трех лекциях мы изучили механику объектно-ориентированного программирования: как объявлять классы, инкапсулировать состояние, выстраивать композитные связи и полиморфные контракты.
Однако знание синтаксиса классов само по себе не защищает от создания катастрофически плохого софта. В индустрии чрезвычайно распространена ситуация, когда код формально написан с использованием ключевого слова class, но архитектурно представляет собой чудовищный монолит — так называемый **Божественный объект (God Object)**.
В начале 2000-х годов Роберт Мартин («Дядя Боб») сформулировал пять принципов объектно-ориентированного дизайна, которые акроним **SOLID** объединил в единую инженерную систему: - **S** — Single Responsibility Principle (Принцип единственной ответственности) - **O** — Open/Closed Principle (Принцип открытости / закрытости) - **L** — Liskov Substitution Principle (Принцип подстановки Лисков) - **I** — Interface Segregation Principle (Принцип разделения интерфейсов) - **D** — Dependency Inversion Principle (Принцип инверсии зависимостей)
Сегодня мы глубоко разберем первые два принципа — **SRP** и **OCP**, а также фундаментальные метрики качества архитектуры: **Связанность (Cohesion)** и **Зацепление (Coupling)**.

План лекции (90 минут)

№	Время	Тема	Формат
1	00:00–00:20	Метрики архитектуры: Внутренняя связанность (Cohesion) и Внешнее зацепление (Coupling)	Архитектурная теория
2	00:20–00:45	Принцип единственной	Разбор антипаттерна

№	Время	Тема	Формат
		ответственности (SRP): определение через акторов изменений	
3	00:45–01:10	Принцип открытости/закрытости (OCP): расширение функционала без модификации кода	Анализ кейсов
4	01:10–01:25	Практический рефакторинг: декомпозиция монолитного процессинга транзакций	Живой код
5	01:25–01:30	Подготовка к СРОП 2 (контрольная точка РК1) и выводы	Рефлексия

---

1. Связанность (Cohesion) и Зацепление (Coupling)

Любое решение в архитектуре программного обеспечения оценивается через баланс двух сил:

Идеал архитектуры
= **High Cohesion** (Высокая связанность)
+ **Low Coupling** (Низкое зацепление)

[НИЗКАЯ СВЯЗАННОСТЬ (Плохо)] [ВЫСОКАЯ СВЯЗАННОСТЬ (Хорошо)]

Класс делает всё подряд: БД, UI, расчеты, отправку SMS.

Все методы класса сфокусированы на одной конкретной цели.

[ВЫСОКОЕ ЗАЦЕПЛЕНИЕ (Плохо)] [НИЗКОЕ ЗАЦЕПЛЕНИЕ (Хорошо)]

Классы намертво привязаны друг к другу через конкретные типы.

Классы взаимодействуют через абстрактные протоколы и интерфейсы.

Cohesion (Внутренняя связанность)

Показывает, насколько тесно методы и поля внутри одного класса объединены единой смысловой задачей. - Если у вас в классе BusValidator есть методы read_nfc(), calculate_fare(), save_to_postgresql() и send_telegram_notification() — у класса катастрофически **низкая связанность**.

Coupling (Внешнее зацепление)

Показывает, насколько сильно изменение в одном модуле вынуждает вносить изменения в другие модули. - Если изменение схемы таблицы в базе данных ломает метод валидации билета в терминале — система страдает от **высокого зацепления**.

2. Single Responsibility Principle (SRP)

Классическая формулировка Роберта Мартина: > **«У класса должна быть одна, и только одна, причина для изменения»** (*A class should have one, and only one, reason to change*). Более строгое и практичное переопределение через заинтересованные стороны (акторов): > **«Модуль должен отвечать перед одним, и только перед одним, актором»** (*A module should be responsible to one, and only one, actor*).

Анализ антипаттерна God Object:

```
# АНТИПАТТЕРН: Нарушение SRP
class BusValidatorGodObject:
    def __init__(self, db_conn):
        self.db = db_conn

    def read_card(self): ...                # Актор: Инженер аппаратного обеспечения (Hardware)
    def calculate_fare(self): ...           # Актор: Финансовый директор / Бухгалтерия (Finance)
    def persist_transaction(self): ...      # Актор: Администратор баз данных (DBA)
    def render_screen_message(self): ...    # Актор: UI/UX дизайнер интерфейса (Design)
```

Если финансовый отдел меняет тарифную сетку — разработчик меняет класс BusValidatorGodObject. Если производитель валидатора сменил чип экрана — разработчик меняет тот же самый файл! В итоге в одном файле неизбежно возникают конфликты слияния в Git (Merge Conflicts), регрессионные ошибки и взаимные поломки.

Решение по SRP:

Разделяем монолит на четыре специализированных класса с высокой связанностью: 1. NfcCardReader — чтение данных с физического ридера. 2. FareCalculator — финансовая логика тарификации. 3. TransactionRepository — сохранение транзакций в хранилище. 4. ValidatorDisplay — отображение графики и световых сигналов.

---

3. Open/Closed Principle (OCP)

Сформулирован Бертраном Мейером в 1988 году: > **«Программные сущности (классы, модули, функции) должны быть открыты для расширения, но закрыты для модификации»** (*Open for extension, closed for modification*).

- **Открыты для расширения**: поведение системы можно дополнять новым функционалом при появлении новых бизнес-требований.
- **Закрыты для модификации**: добавление нового функционала не должно требовать переписывания или изменения существующего, протестированного рабочего кода.

Как добиться OCP на практике?

Главный инструмент OCP — **абстракция и полиморфизм**.

До OCP (Нарушение принципа):

```
# При добавлении нового типа оплаты (например, Kaspi QR)
# мы ВЫНУЖДЕНЫ МОДИФИЦИРОВАТЬ рабочий метод!
class PaymentProcessor:
    def process(self, payment_type: str, amount: float):
        if payment_type == "CARD":
```

```
            self._process_card(amount)
        elif payment_type == "QR":
            self._process_qr(amount)
        elif payment_type == "CASH": # Модификация исходного кода!
            self._process_cash(amount)
```

После рефакторинга по OCP:

```
# domain/payment_processor.py
from typing import Protocol
from decimal import Decimal


class PaymentMethod(Protocol):
    """Абстракция платежного метода (закрыта для изменений)."""
    def execute_payment(self, amount: Decimal) -> bool:
        ...


class PaymentProcessor:
    """Процессор открыт для новых методов, но закрыт от правок."""
    def process(self, method: PaymentMethod, amount: Decimal) -> bool:
        return method.execute_payment(amount)
```

Теперь, когда в систему добавляется оплата по Bluetooth, мы создаем новый файл bluetooth_payment.py с новым классом, реализующим PaymentMethod. **Класс PaymentProcessor не меняется ни на одну букву!** Риск регрессии равен нулю.

---

4. Практический разбор рефакторинга монолита

Рассмотрим архитектурный каркас правильного терминала валидатора:

```
# validator/core.py
from decimal import Decimal
from typing import Protocol


class CardReader(Protocol):
    def read_card_id(self) -> str: ...


class FareEngine(Protocol):
    def compute_fare(self, card_id: str) -> Decimal: ...


class TransactionLedger(Protocol):
    def record_success(self, card_id: str, fare: Decimal) -> None: ...


class AudioVisualFeedback(Protocol):
    def notify_success(self) -> None: ...
    def notify_failure(self, reason: str) -> None: ...


class TransitValidatorTerminal:
```

```
    """
    Класс-оркестратор: Высокая связанность, низкое зацепление.
    Строго соблюдает SRP (оркестрация процесса) и OCP.
    """
    def __init__(
        self,
        reader: CardReader,
        fare_engine: FareEngine,
        ledger: TransactionLedger,
        feedback: AudioVisualFeedback
    ):
        self._reader = reader
        self._fare_engine = fare_engine
        self._ledger = ledger
        self._feedback = feedback

    def process_tap(self) -> bool:
        try:
            card_id = self._reader.read_card_id()
            fare = self._fare_engine.compute_fare(card_id)
            self._ledger.record_success(card_id, fare)
            self._feedback.notify_success()
            return True
        except Exception as e:
            self._feedback.notify_failure(str(e))
            return False
```

Любой из компонентов (reader, ledger, feedback) может быть заменен (например, на мок-объект в тестах) без малейшего изменения самого терминала!

Контрольные вопросы и выводы

1. В чем разница между метриками Cohesion и Coupling? Почему идеалом является «High Cohesion, Low Coupling»?
2. Как формулируется принцип Single Responsibility через понятие «актора изменений»?
3. Каковы признаки класса с запахом кода «God Object»?
4. Каким образом полиморфизм и абстрактные интерфейсы позволяют реализовать Open/Closed Principle?
5. Почему соблюдение SRP и OCP резко снижает количество конфликтов слияния (Merge Conflicts) в командной разработке?

Литература к лекции

1. **Мартин Р.** Чистая архитектура. – СПб.: Питер, 2021. (Главы 7 и 8: SRP и OCP).
2. **Мартин Р.** Быстрая разработка программ. Принципы, примеры, практика. – М.: Вильямс, 2004.
3. **Фаулер М.** Рефакторинг: улучшение проекта существующего кода. – М.: Диалектика, 2020. (Глава 6: Составление методов — Extract Class).